

Cours Linux

L1 MI - UE Linux

Département informatique

Université d'Angers

Juillet 2024



**FACULTÉ
DES SCIENCES**
*Unité de formation
et de recherche*

Plan

1 UE Linux

- Programme

2 CM LINUX

- Système de fichiers
- Commandes fondamentales
- Redirections, filtres et tubes
- Recherche de texte et de fichiers
- Compléments

CM UE : Programme

Programme

Linux

- 4 CM de 1h20
- 6 TP de 1h20
- 1 CT de 1h30
- 1 CT seconde chance de 1h30

Groupes CM

5 groupes de CM - 4 séances de 1h20

- Groupe DL MI : Nicolas Dupin
- Groupe MI 1 : Jean-Michel Richer
- Groupe MI 2 : David Lesaint
- Groupe MI 3 : Olivier Goudet
- Groupe MI 4 : Claudia Vasconcellos

Groupe TP

9 groupes de TP - 6 séances de 1h20

- Groupe DL MI : Nicolas Dupin
- Groupe MI 1A : Claudia Vasconcellos
- Groupe MI 1B : Victor Allard / Ivanhoé Botcazou
- Groupe MI 2A : David Lesaint
- Groupe MI 2B : Jean-Michel Richer
- Groupe MI 3A : Olivier Goudet
- Groupe MI 3B : Quentin Suire
- Groupe MI 4A : Victor Allard
- Groupe MI 4B : Ivanhoé Botcazou

Supports

Espace Moodle : n° 20893, clé 3562pw

- Groupe, calendrier, planning détaillé
- CM : livret (diaporama) + annexes
- TP : livret des énoncés + annexes + corrections
- CT : énoncés + annexes + dépôts

CM Linux : Système de fichiers

Le système d'exploitation

Il est à l'interface entre **l'utilisateur et le matériel**

Ses principales fonctions sont :

- Contrôle des **ressources** (allocation et gestion du processeur et de la mémoire)
- Contrôle des **processus**
- Contrôle des **périphériques** ...

Il contient des outils de gestion utilisables par les applications, tels que la manipulation de fichier, la gestion de l'impression, la date et l'heure

Composants d'un système d'exploitation

Un système d'exploitation est un logiciel volumineux et complexe, découpé en modules par groupes de fonctions, sous-systèmes ou composants

Interpréteurs de commandes (terminaux, ou shells)		
Commandes, compilateurs, bibliothèques		
Interface d'appels systèmes pour le noyau		
Systèmes de fichiers	Gestion de la mémoire	Systèmes des processus
Gestion des E/S	Gestion des interruptions	Communica° interprocessus
Gestion des terminaux	Gestion des événements	
Interface noyau pour le matériel		
Contrôleurs	Contrôleurs de périphériques	Contrôleurs
de terminaux	disques, imprimantes etc	de réseaux

Linux

A l'origine de Linux

- Association du projet GNU et du noyau Linux
- L'objectif était d'associer UNIX à une interface graphique pour les PC
- Implémentation indépendante et conforme à POSIX
- Le noyau d'origine a été écrit par Linus Torvalds, étudiant en informatique à l'Université d'Helsinki

Distributions de Linux

- **Debian**
- Ubuntu, Suse, Fedora, RedHat, Knoppix, Slackware, Mandriva

Avantages de Linux

Pourquoi est-ce que Linux est utilisé au département d'informatique ?

- Multi-tâches
- Multi-utilisateurs
- Multi-postes
- Libre et gratuit (contrairement à Windows et Mac OS)

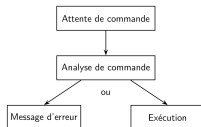
Linux est un environnement pratique

- Pour exécuter en une ligne des commandes efficaces
- Pour lancer des scripts effectuant un grand nombre de tâches
- Pour le développement logiciel (édition, compilation, distribution ...)
- C'est le système d'exploitation utilisé sur les serveurs de calcul haute performance

Interpréteur de commandes

Programme qui analyse et exécute les commandes ligne par ligne

- Appelé **shell** sous Linux avec lequel on interagit via un **terminal**
- Nombreuses familles de shell : sh, jsh, ksh, **bash**, csh, tcsh, zsh



Ne fait pas partie de l'OS

- Pour l'OS, c'est un programme comme un autre : on peut remplacer un interpréteur de commandes par un autre
- Pour les utilisateurs, l'interpréteur est un moyen d'accéder aux services de l'OS

Initiation au shell

Une fois connecté

- ❶ Le système reconnaît l'utilisateur
- ❷ Il ouvre une session à son nom
- ❸ Il attend ses instructions par l'intermédiaire du shell

Le shell

- Est l'interface utilisateur standard
- Permet d'interpréter des programmes ([scripts](#))

Les commandes doivent respecter certaines règles

- Le shell a sa syntaxe : c'est un langage
- Ce n'est pas un langage de programmation compilé comme C++
- C'est un langage de [commandes](#) uniquement

Ouvrir une session Linux

Le serveur starwars du département d'informatique

- Vous y disposez d'un répertoire personnel (home directory) accessible avec votre identifiant (user id) et mot de passe ENT/LDAP
- S'y connecter depuis les postes de nos salles PC ou bien à distance avec la commande `ssh` (cf. Compléments)

Pour tester les exemples de commandes donnés par la suite

- 1 Téléchargez l'archive **cm-linux-1.tgz** placée sur Moodle
- 2 Dans votre répertoire personnel, décompressez l'archive :
`tar xvf cm-linux-1.tgz`
- 3 Déplacez-vous dans le répertoire qui vient d'être créé :
`cd cm-linux-1`

Format des commandes

Syntaxe

```
commande [-option(s)] [argument(s)]
```

Exemple : 0 option et 0 argument

```
date
```

Exemple : 0 option et 2 arguments

```
cp fichier1 fichier2
```

Exemple : 2 options et 0 arguments

```
ls -l -a ou ls -la
```

Exemple : 2 options et 2 arguments

```
cp -Ri dossierSource dossierCible
```

Commandes d'aide ... aux commandes

- Lister toutes les pages du manuel faisant référence à `mot_clé`

`man -k mot_clé`

- Savoir en une ligne à quoi sert une commande

`whatis commande`

- Lire la page du manuel sur la commande

`man commande`

- Obtenir de l'aide en ligne sur une commande

`commande --help`

- Localiser le fichier exécutable correspondant à une commande

`which commande`

Exécution de commandes au premier plan

Quand une commande est exécutée

- ❶ Le shell lance le programme correspondant
 - L'OS crée un nouveau **processus**
- ❷ Le shell attend (il est inutilisable) pendant toute l'exécution du programme
 - Le programme lancé est exécuté **au premier plan**
- ❸ On peut interrompre le programme en tapant **<Ctrl> + C**

On peut lancer deux commandes à suivre en les séparant par un point-virgule ;

```
date; cp fichier1 fichier2
```

Exécution de commandes en arrière plan

Exécution en arrière plan (en mode détaché ou encore en asynchrone)

- En ajoutant **une esperluette &** à la fin de la commande
- On peut lancer d'autres commandes avant qu'elle ne se termine

```
$ sleep 5 &
```

```
[1] 14313 -> le job 1 de la commande correspond au processus 14313
```

```
$ date
```

```
Mon Dec 7 10:40:33 CET 2020 -> résultat de la commande date
```

```
$
```

```
[1]+ Done sleep 5 -> la commande sleep vient de se terminer
```

```
$
```

La commande **ps** donne la liste des processus lancés à partir du terminal

Utiliser efficacement le terminal

Récupération de commandes

- Consultez l'historique des commandes avec `history`
- Réexécutez la dernière commande en tapant `!!`
- Réexécutez la 123^{eme} commande en tapant `!123`
- Rappelez les commandes antérieures avec la `flèche du haut`

Système de fichiers

Le système de fichiers UNIX contient des fichiers et des répertoires

- Un répertoire peut contenir des fichiers et d'autres répertoires

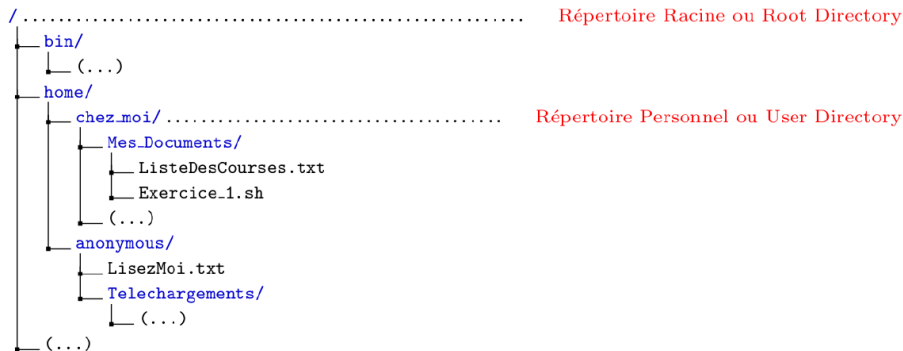
Le système de fichiers UNIX est arborescent

- Ses noeuds internes sont les répertoires
- Ses feuilles sont les fichiers
- Le répertoire racine est noté `/`

Un disque logique correspond à un sous-arbre

- Qui est monté dans le système de fichiers automatiquement au démarrage
- Ou lorsqu'on connecte un support externe de format compatible (clé USB, ...) : le sous-arbre est situé dans le répertoire `/mnt`

Exemple d'arborescence de fichiers Linux



Les principaux répertoires et leur contenu

Répertoire	Contenu
/	Répertoire racine : Toutes les données accessibles par le système
/home	Les répertoires personnels des utilisateurs
/bin	Binaires exécutables des commandes de bases (cd, ls, mkdir, ?)
/lib	Librairies partagées et modules du noyau
/usr	Ressources accessibles par les utilisateurs
/etc	Fichiers de configuration (profile, passwd, fstab...)
/tmp	Données temporaires
/dev	Fichiers spéciaux correspondants aux périphériques
/mnt	Points de montage des périphériques
/var	Fichiers de log ou fichiers changeant fréquemment
/root	Répertoire personnel de l'administrateur

Chemins de fichiers/répertoires

Il existe un chemin reliant tout noeud de l'arborescence à tout autre

- Pour un même fichier/répertoire cible, le chemin diffère selon le répertoire de départ

On parle de chemin absolu ou chemin relatif selon le répertoire de départ

- **Chemin absolu** : on part de la racine de l'arborescence
- **Chemin relatif** : on part du répertoire courant

Le répertoire courant est le répertoire d'où sont lancées les commandes

- Par défaut, c'est le répertoire personnel (home directory)
- Mais on peut le modifier en se déplaçant (par commandes) dans l'arborescence de fichiers

Syntaxe des chemins

Un chemin se note par sa séquence (de noms) de répertoires séparés deux à deux par le symbole slash /

Certains répertoires spéciaux sont dénotés par un symbole particulier

- Répertoire racine /
- Répertoire courant .
- Répertoire parent du répertoire courant ..
- Répertoire personnel ~

Chemins absolus

Le chemin absolu d'un fichier/répertoire correspond au chemin partant de la racine de l'arborescence

```
/ ..... Un chemin absolu part de la racine /  
├── home/ ..... /home/  
│   ├── chez_moi/ ..... /home/chez_moi/  
│   │   ├── Etoiles/ ..... /home/chez_moi/Etoiles/  
│   │   │   ├── SOLEIL.jpg ..... /home/chez_moi/Etoiles/SOLEIL.jpg  
│   │   │   ├── Soleil.jpg ..... /home/chez_moi/Etoiles/Soleil.jpg  
│   │   └── Systeme_Solaire/ ..... /home/chez_moi/Systeme_Solaire/  
│   │       └── SOLEIL.jpg ..... /home/chez_moi/Systeme_Solaire/SOLEIL.jpg
```

Chemins relatifs

Le chemin relatif d'un fichier/répertoire correspond au chemin partant du répertoire courant

- Il est relatif car dépendant du répertoire courant

```
home/..... ../..
└─ chez_moi/..... ../
    └─ Etoiles/..... Répertoire Courant ./
        └─ SOLEIL.jpg..... SOLEIL.jpg ou ./SOLEIL.jpg
            └─ Antares.jpg..... Antares.jpg ou ./Antares.jpg
        └─ Systeme_Solaire/..... ../Systeme_Solaire/
            └─ terre.gif..... ../Systeme_Solaire/terre.gif
```

Le langage des chemins

Multiplés façons de désigner un fichier/répertoire cible

- Chemin absolu ou chemin relatif
- Avec ou sans symboles de répertoires spéciaux

```

/..... Répertoire Racine
├── home/
│   ├── chez_moi/ ..... Répertoire Personnel
│   │   ├── Etoiles/.... Répertoire Courant
│   │   │   └── Soleil.jpg..... Fichier cible

```

Chemins Absolus

```

/home/chez_moi/Etoiles/Soleil.jpg
~/Etoiles/Soleil.jpg
/home/chez_moi/../../chez_moi/Etoile/Soleil.jpg
/home/chez_moi/../../home/chez_moi/Etoile/Soleil.jpg

```

Chemins Relatifs

```

Soleil.jpg
../Etoile/Soleil.jpg
../../chez_moi/Etoile/Soleil.jpg

```

Commandes sur les répertoires

Accès au chemin absolu d'un répertoire

- `pwd` donne le chemin du répertoire courant (path working directory)

Changement de répertoire

- `cd` change de répertoire (change directory)

Création et destruction de répertoire

- `mkdir` crée un répertoire (make directory)
- `rmdir` détruit un répertoire vide (remove directory)

Chemins absolus ou relatifs

- Le répertoire en argument des commandes `cd`, `mkdir` et `rmdir` est désigné par un chemin absolu ou relatif

Commandes pwd et cd

```
$ pwd
/home/m/y/my.login/
→ le répertoire courant est notre répertoire personnel

$ cd cm-linux-1/
→ on se situe maintenant dans le répertoire cm-linux-1

$ pwd
/home/m/y/my.login/cm-linux-1

$ cd Etoiles/; pwd
/home/m/y/my.login/cm-linux-1/Etoiles
→ on a enchaîné 2 commandes séparées par le symbole ;
→ on se situe maintenant dans le répertoire Etoiles
→ cd ./Etoiles/ est équivalent à cd Etoiles/ car le point . dénote le répertoire
courant qui contient bien Etoiles

$ cd ~; pwd
/home/m/y/my.login
→ on est revenu dans notre répertoire personnel dénoté par le tilde ~
```

Commandes pwd et cd

```
$ cd ./cm-linux-1/Etoiles/; pwd
```

```
/home/m/y/my.login/cm-linux-1/Etoiles
```

→ on est redescendu dans le répertoire **Etoiles** en une commande

```
$ cd ../Systeme_Solaire/; pwd
```

```
/home/m/y/my.login/cm-linux-1/Systeme_Solaire
```

→ on s'est placé dans le répertoire adelphe **Systeme_Solaire** en remontant d'abord dans le répertoire parent dénoté par le double-point **..**

```
$ cd /home/m/y/my.login/cm-linux-1/Etoiles
```

→ on est revenu dans le répertoire **Etoiles** en spécifiant son chemin absolu

```
$ cd ../../
```

→ où est-on ?

Archivage de répertoires et compression de fichiers

L'archivage d'un répertoire crée un fichier archive en préservant le contenu

- Utile pour les transferts (email, téléversement, téléchargement)
- Sans pertes : on retrouvera le répertoire après désarchivage

La compression d'un fichier crée un fichier de moindre taille

- On combine généralement archivage de répertoire et compression

Archivage/compression avec la commande `tar`

- `-z` : compression gzip
- `-c` : création d'une nouvelle archive
- `-v` : *verbose*, affiche des informations
- `-f` : précède le nom de l'archive à créer

Commande tar : archivage et compression

```
$ mkdir monRep tmp; cp -r Etoiles Système_Solaire monRep; ls monRep
Etoiles Système_Solaire
$ tar -czvf archive.tar.gz monRep
a monRep
a monRep/Etoiles
a monRep/Système_Solaire
a monRep/Système_Solaire/.DS_Store
a monRep/Système_Solaire/terre.gif
a monRep/Etoiles/SOLEIL.jpg
a monRep/Etoiles/.DS_Store
a monRep/Etoiles/Antares.jpg
→ création de l'archive compressée archive.tar.gz du dossier monRep
$ file archive.tar.gz
archive.tar.gz: gzip compressed data, last modified: Tue Dec 8 17:45:10
2020, from Unix, original size modulo 2^32 183296
$ ls -l archive.tar.gz
-rw-rw---- 1 my.login ldapinfo 133693 8 déc 18:45 archive.tar.gz
→ taux de compression 133693 / 183296
```

Extraction d'archives

Décompression et extraction toujours avec la commande `tar`

- `-x` : extraction
- `-C` : répertoire où l'on extrait (par défaut, le répertoire courant)
- `-z` : décompression gunzip
- `-v` : *verbose*, affiche des informations

Commande tar : décompression et extraction

```
$ mv archive.tar.gz tmp/; cd tmp
```

```
$ tar -zxf mon_archive.tar.gz
```

→ extraction dans le répertoire courant

```
$ ls
```

```
archive.tar.gz monRep
```

```
$ mkdir ../tmp1
```

```
$ tar -zxf archive.tar.gz -C ../tmp1
```

→ extraction dans le répertoire **tmp1**

```
$ ls ../tmp1
```

```
monRep
```

CM Linux : Commandes fondamentales

Affichage de répertoires

Afficher le contenu de répertoires avec `ls`

`ls [-algiARF...] [FILE]...`

- Celui du/des répertoire(s) en argument(s)
- Celui du répertoire courant s'il n'y a aucun argument

Quelques options de `ls`

- `-a` tous les fichiers/répertoires y compris les fichiers/répertoires cachés (noms commençant par `.`)
- `-l` affichage au format long
- `-c` ou `-t` tri par dernière date de modification
- `-R` récursif

Commande ls

```
$ ls /usr
```

```
bin games include lib local sbin share src
```

```
$ cd cm-linux-2; ls -l
```

```
total 8 mon.login
```

```
drwxr-xr-x 2 mon.login ldapinfo 4096 Dec  7 12:44 Etoiles
```

```
-rw-r----- 1 mon.login ldapinfo  23 Dec  7 09:36 fichier1
```

```
drwxr-xr-x 2 mon.login ldapinfo 4096 Dec  7 12:44 Système_Solaire
```

→ affiche le listing détaillé du contenu du répertoire courant

```
$ ls -a
```

```
. .. Etoiles fichier1 Système_Solaire .unfichiercache
```

→ affiche les fichiers cachés tel **.unfichiercache**

```
$ ls Etoiles Système_Solaire
```

Etoiles:

Antares.jpg SOLEIL.jpg

Système_Solaire:

terre.gif

Noms de fichiers et extensions

Format des noms de fichiers

- Suite de caractères ASCII ne contenant pas de slashes /
- Longueur limitée (14 ou 255 caractères)
- Pas de contraintes, mais des suffixes conventionnels

Suffixes/extensions propres aux langages de programmation

- | | |
|--|---|
| • <code>.s</code> | Assembleur |
| • <code>.c</code> , <code>.h</code> | C et headers |
| • <code>.java</code> , <code>.class</code> | Java |
| • <code>.f</code> | Fortran |
| • <code>.el</code> , <code>.elc</code> | Lisp (compilé) |
| • <code>.l</code> , <code>.y</code> | Lex et Yacc |
| • <code>.o</code> , <code>.a</code> | Binaire objet, librairie de binaires objets |

Création de fichiers

Différentes possibilités pour créer un nouveau fichier

- En utilisant un éditeur de texte avec interface graphique comme `kate`
- En utilisant un éditeur de texte en mode console comme `nano` ou `vim`
- Avec la commande `touch fic`
 - Crée le fichier vide dénommé **fic**
 - Si le fichier existait déjà, met à jour sa date de dernière modification
- A la volée
 - ① Exécutez la commande `cat > fic`
 - ② Saisissez le contenu du fichier (y compris les retours à la ligne)
 - ③ Pressez deux fois sur `<Ctrl> + D` : le fichier **fic** a été créé avec votre contenu !

Affichage de fichiers

Contenu d'un fichier

- `cat` affiche le contenu d'un fichier
- `head` désigne les premières lignes
- `tail` désigne les dernières lignes
- `more` affiche le fichier
- `less` affiche le fichier (plus avancé que `more`)

Afficher le contenu d'un fichier

- Affichage basique du fichier (utile pour les petits fichiers)

`cat fichier`

- Affichage des lignes du fichier de la dernière à la première

`tac fichier`

- Affichage du contenu du fichier page par page

`more fichier`

- Affichage offrant davantage de possibilités

`less fichier`

- Tapez `/expression` pour rechercher `expression` dans le fichier
- Tapez `n` pour atteindre la prochaine occurrence de `expression`
- Tapez `N` pour revenir à l'occurrence précédente de `expression`
- Tapez `v` pour éditer le fichier (généralement avec l'éditeur nano)

Afficher le contenu d'un fichier

- Affiche les 10 premières lignes de **fichier**

```
head fichier
```

- Affiche les 5 premières lignes de **fichier**

```
head -n 5 fichier
```

- Affiche les 10 premiers octets de **fichier**

```
head -c 10 fichier
```

- Affiche les 10 dernières lignes de **fichier**

```
tail fichier
```

- Affiche les 20 dernières lignes de **fichier**

```
tail -n 20 fichier
```

- Affiche les 7 derniers octets de **fichier**

```
tail -c 7 fichier
```

Copie de fichiers

Copier fichiers et répertoires avec `cp`

```
cp [-isR...] SOURCE DEST
```

```
cp [-isR...] SOURCE... DIRECTORY
```

Trois usages types

```
$ cp f1 f2
```

- Crée le fichier **f2** qui est une copie de **f1**

```
$ cp -R rep1 rep2
```

- Crée le répertoire **rep2** qui est une copie récursive intégrale de **rep1**

```
$ cp fic /home/m/y/my.login/rep/
```

- Copie **fic** dans le répertoire **rep** (qui doit pré-exister)

Destruction de fichiers

Destruction de fichier/répertoire avec rm

`rm [-ifR...] [FILE]...`

- `-i` mode interactif (demande de confirmation)
- `-f` force la destruction
- `-r` destruction récursive (utile pour les répertoires)

Exemple

```
$ mkdir myrm; cd myrm; touch toKeep toDelete1 toDelete2; ls
toDelete1 toDelete2 toKeep
$ rm -i toKeep
rm: remove regular empty file 'toKeep'? n
$ rm -i toDelete1
rm: remove regular empty file 'toDelete1'? y
$ rm -f toDelete2
$ ls
toKeep
```

Déplacement et renommage de fichiers

Déplacement et renommage de fichier/répertoire avec `mv`

```
$ mv file1 file2
```

- Renomme **file1** en **file2**

```
$ mv dir1 dir2
```

- Renomme le répertoire **dir1** en **dir2** (qui ne doit pas pré-exister)

```
$ mv file1 file2 calc.dat ~/dir2
```

- Déplace les fichiers **file1**, **file2** et **calc.dat** dans le répertoire **~/dir2** (qui doit pré-exister)

Filtres/modèles sur noms de fichiers (globs)

De nombreuses commandes (`ls`, `rm` ...) acceptent plusieurs fichiers en arguments

- Plutôt que d'énumérer tous les fichiers désirés, on passe à la commande un **modèle** de noms de fichier qui correspond aux noms de ces fichiers

Certains caractères spéciaux permettent de définir les modèles de noms

? [] - ^ * \

Caractères spéciaux pour modèles de noms de fichiers

? [] - ^ * \

- Le point d'interrogation ?
 - Désigne n'importe quel caractère alphanumérique (mais un seul)
- Les crochets []
 - Désignent un caractère quelconque parmi ceux énumérés
[ch] soit c, soit h
- Le tiret - dans les crochets permet de définir une classe de caractères
[a-e] une lettre entre a et e
[a-zA-Z] une lettre minuscule ou majuscule
[a-z][0-9] une lettre minuscule suivie d'un chiffre
- L'accent circonflexe ^ dans les crochets permet d'exclure des classes
[^d-f] une lettre qui n'est pas entre d et f

Caractères spéciaux pour modèles de noms de fichiers

? [] - ^ * \

- L'astérisque ou étoile `*`
 - Désigne n'importe quelle séquence de caractères alphanumériques y compris la séquence vide
 - A utiliser avec précaution en particulier avec la commande `rm`
 - L'anti-slash `\`
 - Inhibe la signification du caractère spécial qu'il précède : ce caractère est interprété littéralement en ligne de commande
- ```
$ touch abc a?c; ls
abc a?c
$ ls a?c
abc a?c
$ ls a\?c
a?c
```

## Caractères spéciaux : exemples avec la commande `ls`

- Liste tous les fichiers et répertoires sauf ceux qui commencent par un point (les fichiers/répertoires cachés)

```
ls *
```

- Liste tous les fichiers et répertoires dont le nom se termine par un c

```
ls *c
```

- Liste tous les fichiers dont le nom commence par un point

```
ls .*
```

- Liste tous les fichiers dont le nom commence par un chiffre

```
ls [0-9]*
```

- Liste tous les fichiers dont le nom se termine par une majuscule

```
ls *[A-Z]
```

- Liste tous les fichiers dont le nom débute par un a suivi d'un caractère

```
ls a?
```

# Caractères spéciaux : exemples avec la commande `rm`

- Supprime tous les fichiers (sauf les fichiers cachés)

```
rm *
```

- Supprime tous les fichiers et répertoires (sauf les fichiers cachés)

```
rm -r *
```

- Supprime tous les fichiers cachés

```
rm .*
```

- Supprime tous les fichiers dont le nom ne commence pas par une lettre entre a et c

```
rm [^a-c]*
```

- Supprime, en mode interactif et récursif, tous les fichiers et répertoires dont le nom a la lettre e en seconde position

```
rm -ri ?e*
```

# Caractéristiques d'un fichier

Chaque fichier se caractérise par son type et les droits d'accès accordés aux différentes classes d'utilisateurs

- 3 types de fichiers : les fichiers réguliers, les répertoires, et les liens
- 3 classes d'utilisateurs : son propriétaire, son groupe propriétaire, et les autres utilisateurs
- 3 droits sur un fichier : le droit en lecture, en écriture, et en exécution

## Classe d'utilisateurs et droits

- Chaque classe a ses propres droits
- Les droits en lecture écriture, et exécution sont indépendants
- Une classe peut n'avoir aucun droit

# Caractéristiques d'un fichier

Chaque type/classe/droit est dénoté par un caractère particulier

- - : fichier ordinaire (regular file)
- d : répertoire (directory)
- l : lien (link)
- - : aucun droit
- r : droit de lecture (read)
- w : droit d'écriture (write)
- x : droit d'exécution d'un fichier ou de parcours d'un répertoire (execution)
- u : le propriétaire (user)
- g : le groupe (group)
- o : les autres (others)

# Caractéristiques d'un fichier

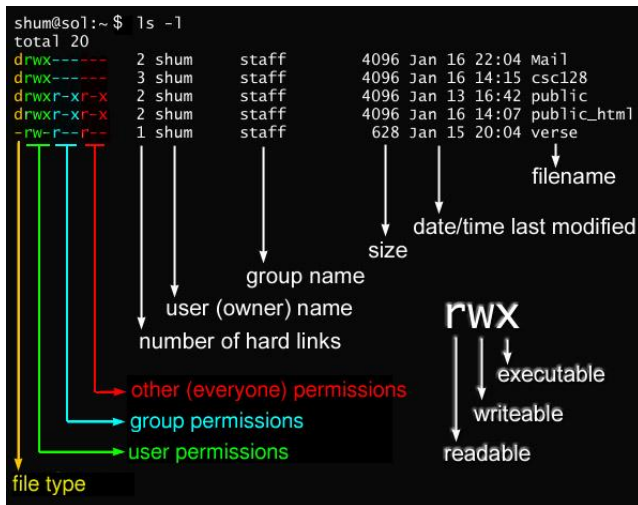
`ls -l` liste les caractéristiques de chaque fichier du répertoire courant sur 10 caractères

- 1 pour le type
- 3 pour le propriétaire : `r/-`, `w/-`, `x/-`
- 3 pour le groupe : `r/-`, `w/-`, `x/-`
- 3 pour les autres : `r/-`, `w/-`, `x/-`

## Exemples

```
-rwxrwxrwx
dr-xr-----
```

# Exemple des caractéristiques d'un fichier



# Changement des droits d'accès

Avec la commande `chmod` en mode symbolique

`chmod [-R] [ugoa] [+-=] [rwx st ugo] FILE...`

- Opérations

- + ajout de droits
- - retrait de droits
- = positionnement de droits

- Classe

- a appliqué à tous (par défaut)
- u appliqué au propriétaire
- g appliqué au groupe
- o appliqué aux autres

- Droits

- r, w et x pour lecture, écriture et exécution
- s set-uid bit (associé à u)
- t sticky bit

# Commande chmod

```
$ ls -l fichier1
-rw-r----- 1 my.login ldapinfo 23 Dec 7 09:36 fichier1
$ chmod a+x,g-r fichier1
$ ls -l fichier1
-rwx--x--x 1 my.login ldapinfo 23 Dec 7 09:36 fichier1
→ fichier1 est lisible par le propriétaire uniquement
→ fichier1 est modifiable par le propriétaire uniquement
→ fichier1 est exécutable par tous
$ chmod go+r,u-w fichier1
$ ls -l fichier1
-r-xr-xr-x 1 my.login ldapinfo 23 Dec 7 09:36 fichier1
→ fichier1 est lisible et exécutable par tous mais non modifiable
$ chmod ug=rw fichier1
$ ls -l fichier1
-rw-rw-r-x 1 my.login ldapinfo 23 Dec 7 09:36 fichier1
→ les droits du propriétaire et du groupe ont été forcés à lecture et écriture
```

# Changement des droits d'accès

Avec la commande `chmod` en mode octal

`chmod octalmode FILE...`

- Les 8 combinaisons possibles de droits pour une classe peuvent être représentées par un nombre octal
- Par convention, on fixe **r**=4, **w**=2 et **x**=1 :

| Octal | Decimal   | Permission             | Representation |
|-------|-----------|------------------------|----------------|
| 000   | 0 (0+0+0) | No Permission          | ---            |
| 001   | 1 (0+0+1) | Execute                | --x            |
| 010   | 2 (0+2+0) | Write                  | -w-            |
| 011   | 3 (0+2+1) | Write + Execute        | -wx            |
| 100   | 4 (4+0+0) | Read                   | r--            |
| 101   | 5 (4+0+1) | Read + Execute         | r-x            |
| 110   | 6 (4+2+0) | Read + Write           | rw-            |
| 111   | 7 (4+2+1) | Read + Write + Execute | rxw            |

L'option `-R` change le mode de tous les sous-répertoires récursivement

# Changement des droits d'accès

3 nombres octaux suffisent à représenter les permissions d'un fichier

| d           | r                 | w     | x    | r                 | -     | x    | r                | -     | -    |
|-------------|-------------------|-------|------|-------------------|-------|------|------------------|-------|------|
|             | read              | write | exec | read              | write | exec | read             | write | exec |
| File type   | Owner permissions |       |      | Group permissions |       |      | User permissions |       |      |
| (directory) | 4                 | 2     | 1    | 4                 | 2     | 1    | 4                | 2     | 1    |
|             | 7                 |       |      | 5                 |       |      | 4                |       |      |

- 400 Lisible par le propriétaire
- 200 Modifiable par le propriétaire
- 100 Exécutable par le propriétaire
- 40 Lisible par les membres du groupe propriétaire
- ...
- 1 Exécutable par les autres utilisateurs

# CM Linux : Redirections, filtres et tubes

# Entrée et sortie standards

Linux fournit trois canaux d'entrée/sortie

- Entrée standard : le clavier par défaut
- Sortie standard : la fenêtre du terminal par défaut
- Erreur standard : la fenêtre du terminal par défaut

Lorsque l'on tape la commande `echo hello world`, le texte `hello world` est ici entré au clavier, puis lorsque la commande est exécutée, ce même texte est affiché à l'écran

- On souhaite parfois rediriger le résultat d'une commande vers un fichier de façon par exemple à sauvegarder ce résultat
- On peut aussi rediriger le flux d'entrée si par exemple on veut appliquer une commande sur le contenu d'un fichier

# Redirection des entrées/sorties standard

Une redirection consiste à créer un canal de communication entre un fichier et un processus à l'aide des caractères spéciaux

< > >> 2> 2>> &>

avec la syntaxe

commande **CARACTERE** nomdefichier

- < redirige la lecture des données de la commande depuis un fichier
- > redirige la sortie de la commande vers un fichier
- >> redirige la sortie de la commande vers la fin du fichier
- 2> redirige la sortie d'erreur vers un fichier
- 2>> redirige la sortie d'erreur vers la fin du fichier
- &> redirige la sortie standard et la sortie d'erreur standard vers un fichier

# Exemples de redirection de flux entrée/sortie

- Ecriture de hello dans un fichier **test.txt**

```
echo hello > test.txt
```

Dans ce cas, le texte hello ne s'affiche plus à l'écran mais est écrit dans le fichier **test.txt**

- Ecriture du résultat de la commande `ls -l` à la fin du fichier `test.txt`

```
ls -l >> test.txt
```

- Ecriture de l'erreur renvoyée par la commande `rm *` dans un fichier **error.txt**

```
rm * 2> error.txt
```

Si le répertoire courant contient un répertoire nommé `dir`, le fichier **error.txt** contiendra le message d'erreur :

```
rm: cannot remove 'dir': Is a directory
```

Les fichiers du répertoire courant seront par contre bien supprimés

# Les filtres

Un filtre est un processus (commande) qui lit les données sur l'entrée standard, modifie ou traite éventuellement ces données et produit des données sur la sortie standard.

Les principaux filtres sont :

- `sort` Pour trier les lignes d'un fichier
- `cut` Pour extraire certains éléments d'un fichier
- `tr` Remplace des caractères dans un fichier
- `uniq` Supprime les lignes adjacentes identiques

# Les commandes des filtres courants – sort

La commande `sort` permet de trier des lignes d'un fichier. Par défaut, le tri est dans l'ordre lexicographique croissant basé sur le code ASCII (notamment, les chiffres précèdent les majuscules qui précèdent les minuscules).

- `-n` trier par ordre numérique (sinon 12 est avant 7)
- `-r` inverser l'ordre de tri
- `-o` écrire le résultat dans un fichier
- `-k` trier sur un champ particulier (par défaut le premier)
- `-t` spécifier le délimiteur de champs (appelé aussi séparateur). Par défaut, le délimiteur est la tabulation ou l'espace.

# Les commandes des filtres courants – sort

## Exemple 1

```
$ more file1.txt
```

Je pense

Donc je suis

→ Cette commande `more` permet simplement de lire le contenu du fichier **file1.txt**.

```
$ sort file1.txt
```

Donc je suis

Je pense

→ La commande `sort` a pris en entrée le contenu du fichier, a trié les lignes du flux entrant en ordre lexicographique puis a affiché le résultat sur la console (sortie standard).

→ Attention le contenu du fichier d'origine n'a pas changé. Les phrases qui étaient présentes dans le fichier **file1.txt** restent dans l'ordre d'origine.

```
$ more file1.txt
```

Je pense

Donc je suis

→ On voit que le contenu du fichier **file1.txt** n'a pas changé.

# Les commandes des filtres courants – sort

## Exemple 2

```
$ more file2.txt
user1:jean:dupont:37
user2:marie:jeanne:46
user3:marie:delattre:101
user4:marc:rolland:46
user5:etienne:voisin:22

$ sort -t ':' -k2 file2.txt
user5:etienne:voisin:22
user1:jean:dupont:37
user4:marc:rolland:46
user2:marie:jeanne:46
user3:marie:delattre:101
```

→ Dans ce cas les lignes du fichier sont triées suivant le deuxième champ si on tient compte du caractère : comme délimiteur de champs.

# Les commandes des filtres courants – sort

## Exemple 2

```
$ sort -t ':' -k2,3 file2.txt
```

```
user5:etienne:voisin:22
```

```
user1:jean:dupont:37
```

```
user4:marc:rolland:46
```

```
user3:marie:delattre:101
```

```
user2:marie:jeanne:46
```

→ Avec l'option `-k2,3` on a fait un tri suivant le deuxième champ, puis selon le troisième en cas d'égalité suivant le deuxième champ (cas des users avec le même prénom marie).

```
$ sort -t ':' -k4 file2.txt
```

```
user3:marie:delattre:101
```

```
user5:etienne:voisin:22
```

```
user1:jean:dupont:37
```

```
user2:marie:jeanne:46
```

```
user4:marc:rolland:46
```

→ Voici un tri suivant le quatrième champ. Est-ce que vous remarquez un problème ?

# Les commandes des filtres courants – sort

```
$ sort -nt ':' -k4 file2.txt
```

```
user5:etienne:voisin:22
```

```
user1:jean:dupont:37
```

```
user2:marie:jeanne:46
```

```
user4:marc:rolland:46
```

```
user3:marie:delattre:101
```

→ Avec l'option `-n` on a maintenant bien un tri par ordre numérique.

```
$ sort -rnt ':' -k4 file2.txt > output.txt
```

→ Que fait cette commande selon vous ?

# Les commandes des filtres courants – cut

La commande `cut` permet d'extraire des colonnes ou des champs des lignes d'un fichier ou de l'entrée standard.

- `-c` extraire suivant le nombre de caractères
- `-f` extraire suivant le nombre de champs
- `-d x` le caractère `x` est le délimiteur de champ (souvent `,` ou `;`). Le délimiteur par défaut est la tabulation.

# Les commandes des filtres courants – cut

## Exemple 1 : extraction d'un caractère

```
$ more file2.txt
user1:jean:dupont:37
user2:marie:jeanne:46
user3:marie:delattre:101
user4:marc:rolland:46
user5:etienne:voisin:22
```

```
$ cut -c 7 file2.txt
```

```
j
m
m
m
e
```

→ On extrait ici le septième caractère de chaque ligne.

# Les commandes des filtres courants – cut

## Exemple 2 : extraction d'une série de caractères

```
$ cut -c 4-6 file2.txt
```

```
r1:
```

```
r2:
```

```
r3:
```

```
r4:
```

```
r5:
```

→ On extrait les caractères de 4 à 6 de chaque ligne.

```
$ cut -c 4,9 file2.txt
```

```
ra
```

```
rr
```

```
rr
```

```
rr
```

```
ri
```

→ Extraction des caractères 4 et 9 de chaque ligne.

# Les commandes des filtres courants – cut

## Exemple 3 : extraction de champs

```
$ cut -f 2,3 -d ':' file2.txt
```

```
jean:dupont
marie:jeanne
marie:delattre
marc:rolland
etienne:voisin
```

→ Extraction des champs 2 et 3 suivant le délimiteur :

```
$ cut -f 1-3 -d ':' file2.txt
```

```
user1:jean:dupont
user2:marie:jeanne
user3:marie:delattre
user4:marc:rolland
user5:etienne:voisin
```

→ Extraction des champs de 1 à 3 suivant le délimiteur :

# Filtre tr

```
tr [option] chaine1 chaine2
```

- **tr** signifie “translate characters”
- Il remplace, supprime ou compresse les caractères sélectionnés de chaine1
- Un caractère de chaine1 est remplacé par le caractère de même position dans chaine2
- Principales options de tr :
  - **-c** (*Complément*) Sélectionne tout caractère hors de chaine1
  - **-d** (*Delete*) Suppression des caractères sélectionnés
  - **-s** (*Squeeze*) Remplace aaa par a si chaine1 contient a
- Abréviations
  - **[a-z]** Plage de 26 caractères allant de a à z
  - **[a\*n]** Désigne la chaîne constituée de n fois le caractère a

# Les commandes des filtres courants – tr

## Exemples

```
$ tr 'e' 'a' < file1.txt
```

Ja pansa

Donc ja suis

→ On affiche ici le contenu de **file1.txt** après avoir remplacé les lettres e par des a.

```
$ tr -C 'e' '-' < file1.txt
```

-e-e-e-----e---

→ On affiche ici le contenu de **file1.txt** après avoir remplacé tout caractère différent de e par un -. Notez que le retour à la ligne \n à été remplacé.

```
$ more file3.txt
```

ddaaaaddd777

```
$ tr -s '[a-z]' < file3.txt
```

dad777

→ Toute séquence de minuscules identiques est remplacée par cette minuscule.

# Les commandes des filtres courants – tr

```
$ tr -s '[a-z]' '[A-Z]' < file3.txt
```

DAD777

→ Squeeze des minuscules suivi d'une conversion des lettres en majuscule.

```
$ tr -d 'a' < file3.txt > output.txt
```

→ La lettre a est supprimée puis la sortie de la commande est sauvegardée dans un nouveau fichier **output.txt**.

# Filtre uniq

Le filtre `uniq` retourne un seul exemplaire des lignes adjacentes identiques

- Options :
  - `-c` Compte le nombre d'exemplaires de chaque ligne
  - `-d` Affiche uniquement les lignes adjacentes dupliquées
  - `-i` Ne distingue pas les majuscules des minuscules
  - `-u` Seules les lignes en un seul exemplaire sont affichées

## Exemples d'utilisation de uniq

```
$ more file4.txt
```

```
une pierre
un raton laveur
un raton laveur
deux maisons
une pierre
```

```
$ uniq file4.txt
```

```
une pierre
un raton laveur
deux maisons
une pierre
```

→ une des lignes doublon un raton laveur est supprimée. Les lignes une pierre ne sont pas adjacentes, elles sont conservées.

# Exemples d'utilisation de uniq

```
$ uniq -c file4.txt
```

```
1 une pierre
2 un raton laveur
1 deux maisons
1 une pierre
```

```
$ uniq -d file4.txt
```

```
un raton laveur
```

```
$ uniq -u file4.txt
```

```
une pierre
deux maisons
une pierre
```

# Les tubes de communication

Un tube est un canal de communication entre deux processus.  
Les tubes sont utilisés pour relier deux commandes entre elles.

```
commande1 pipe commande2
```

- | La sortie de `commande1` sera utilisée comme entrée de `commande2`
- ; `commande2` est lancée après la fin de l'exécution de `commande1`

# Exemples d'enchaînement de commandes

```
$ more file2.txt
```

```
user1:jean:dupont:37
```

```
user2:marie:jeanne:46
```

```
user3:marie:delattre:101
```

```
user4:marc:rolland:46
```

```
user5:etienne:voisin:22
```

```
$ cut -f 2 -d ':' file2.txt | uniq
```

```
jean
```

```
marie
```

```
marc
```

```
etienne
```

→ On a d'abord extrait le deuxième champ selon le séparateur : du fichier **file2.txt**, puis on a conservé un seul exemplaire des lignes adjacentes identiques.

# Exemples d'enchaînement de commandes

```
$ sort -rt ':' -k1 file2.txt | cut -f 2,3 -d ':' | tr ':' ' '
```

etienne voisin

marc rolland

marie delattre

marie jeanne

jean dupont

→ On a d'abord trié les lignes suivant un ordre inverse et selon le premier champ, puis on a extrait les champs 2 et 3, puis on a remplacé le caractère : par une espace.

```
$ cut -f 2 -d ':' file2.txt | uniq -u | sort
```

etienne

jean

marc

→ On a d'abord extrait le deuxième champ selon le séparateur : du fichier **file2.txt**, puis on a conservé seulement les lignes présentes en un seul exemplaire et enfin on a effectué un tri selon l'ordre lexicographique.

# Exemples d'enchaînement de commandes

```
$ sort -n -k4 file2.txt | cut -c 7 | tr '[a-z]' '[A-Z]' | uniq >
output.txt
```

→ On a d'abord trié les lignes suivant le 4ème champ avec un ordre numérique, puis on a extrait le 7ème caractère de chaque ligne, puis on a remplacé les lettres minuscules par des majuscules, puis on a conservé un seul exemplaire des lignes adjacentes identiques et enfin on a sauvegardé le résultat dans le fichier **output.txt**.

→ Que contient ce fichier ?

```
$ cut -d ':' -f 2 file2.txt > output1.txt; tr '[a-z]' '[A-Z]' <
output1.txt | uniq -u
```

JEAN

MARC

ETIENNE

→ On a extrait le deuxième champ du fichier qu'on a sauvegardé dans un fichier **output1.txt**. Une fois la commande terminée, on a récupéré le contenu du fichier **output1.txt**, puis on a converti les lettres minuscules en majuscules avant de garder les lignes sans doublons.

→ On peut afficher le même contenu avec la commande `cut -d ':' -f 2 file2.txt | tr '[a-z]' '[A-Z]' | uniq -u` mais le fichier **output1.txt** ne sera pas créé dans ce cas

# CM Linux : Recherche de texte et de fichiers

# Les expressions régulières

Une expression régulière (alias regex, pattern, motif, ...) est une suite de caractères décrivant un ensemble de chaînes de caractères que l'on souhaite rechercher dans un texte (code source, document texte, log, email ...)

## Exemples de recherches

- Les chaînes Section et Sous-section
- Les adresses e-mails de nom de domaine univ-angers.fr
- Tout mot en fin de phrase ponctuée par un point
- Les mots-clés for while do break
- Un lipogramme (texte où une lettre donnée n'apparaît pas)
- ...

# Utilisation des expressions régulières

Pour décrire des langages formels, en particulier dans l'analyse et la manipulation des langages informatiques

- Les compilateurs et interprètes de langages de programmation se basent sur les regex

Pour trouver des blocs de texte documentaire et appliquer un traitement (ajout, modification, remplacement, suppression)

- Les éditeurs de texte et environnements de développement intégrés (EDI) permettent d'exploiter les regex

De nombreuses commandes Linux les utilisent nativement

- Les commandes `grep`, `sed`, ...
- Comme vu précédemment, les shells utilisent un système distinct et moins expressif (glob patterns) qui est basé sur les caractères spéciaux

# La commande `grep`

La commande `grep` (globally search a regular expression and print) recherche et affiche des lignes dans un/des fichiers selon divers critères

`grep [OPTION] PATTERN [FILE] ...`

- `grep` utilise les regex pour définir le modèle des lignes à rechercher
- `grep` affiche les lignes retenues (par défaut)

Il existe 2 syntaxes quasi-identiques des regex

- La syntaxe basique : Basic Regular Expressions
- La syntaxe étendue : Extended Regular Expression

Nous utiliserons la syntaxe étendue - moins verbeuse et plus sûre -

- Avec la commande `grep` en activant l'option `-E`
- Ou, de manière équivalente, avec la commande `egrep`

# Le langage des expressions régulières

Une regex s'exprime comme une suite de 0, 1 ou plusieurs

- Caractères spéciaux (alias méta-caractères, opérateurs) de substitution, groupement et quantification
- Caractères littéraux qui ne représentent qu'eux-mêmes
  - On préfixe ("échappe") un méta-caractère par l'antislash `\` pour qu'il soit interprété littéralement
- Exemple : `[rR]eg(ular expression)|(ex)`

Il y a correspondance (matching) entre une chaîne de caractères et une regex lorsque tout ou partie de la chaîne correspond exactement à l'une de celles décrites par la regex

- Exemple : la regex ci-dessus reconnaît toute chaîne comprenant `regular expression`, `Regular expression`, `regex` ou `Regex`

# Expressions régulières : les méta-caractères

- `.` : se substitue à n'importe quel caractère (un seul)
- `[liste]` : désigne un des caractères de la liste
- `[^liste]` : désigne un caractère hors de la liste

```
$ cat tg.txt
Il fait beau
il fait chaud
dans cette ville.
$ egrep 'ai' tg.txt
Il fait beau
il fait chaud
$ egrep 'a.' tg.txt
Il fait beau
il fait chaud
dans cette ville.
```

```
$ egrep 'e\.' tg.txt
dans cette ville.
$ egrep '[de]a' tg.txt
Il fait beau
dans cette ville.
$ egrep '[^def]a' tg.txt
il fait chaud
$ egrep '[ed]a[nu]' tg.txt
Il fait beau
dans cette ville.
```

# Expressions régulières : les caractères d'espacement

- `<Space>` : désigne une espace
- `<Ctrl> V + <Tab>` : désigne une tabulation
- `\s` : désigne une espace ou une tabulation

```
$ egrep ' ' tg.txt
Il fait beau
il fait chaud
$ egrep ' ' tg.txt
dans cette ville.
$ egrep '\s' tg.txt
Il fait beau
il fait chaud
dans cette ville.
```

# Expressions régulières : les classes de caractères

- `[a-z]` ou `[[:lower:]]` : désigne une minuscule
- `[A-Z]` ou `[[:upper:]]` : désigne une majuscule
- `[a-Z]` ou `[[:alpha:]]` : désigne une lettre
- `[0-9]` ou `[[:digit:]]` : désigne un chiffre
- `[[:alnum:]]` : désigne tout caractère alphanumérique
- `[[:blank:]]` ou `[[:space:]]` : désigne tout caractère d'espacement
- `[[:punct:]]` : désigne tout caractère de ponctuation `] [ ! " # $ % & ' ( ) * + , . / : ; < = > ? @ \ ^ _ ` { | } ~ -`

```
$ egrep '[A-Z]' tg.txt
Il fait beau
$ egrep '[t-v]' tg.txt
Il fait beau
il fait chaud
dans cette ville.
```

```
$ egrep '[[:punct:]]' tg.txt
dans cette ville.
$ egrep '[[:alpha:]][[:punct:]]' tg.txt
dans cette ville.
$ egrep '[[:alpha:]][[:punct:]]' tg.txt
Il fait beau il fait chaud
dans cette ville.
```

# Expressions régulières : concaténation, alternative, isolation

- (e) : forme une expression à partir de l'expression e
- e1e2 : désigne la concaténation des expressions e1 et e2
- | : désigne soit l'expression qui précède, soit celle qui suit

```
$ egrep 'c((ette)|(haud))' tg.txt
il fait chaud
dans cette ville.
$ egrep '(l fait)|(c(ette)|(haud))' tg.txt
Il fait beau
il fait chaud
dans cette ville.
$ egrep 'l ((fait)|(c(ette)|(haud)))' tg.txt
Il fait beau
il fait chaud
```

# Expressions régulières : les quantificateurs

- `?` : désigne l'expression qui précède ou la chaîne vide
- `+` : désigne au moins 1 répétition de l'expression qui précède
- `*` : désigne 0 ou plusieurs répétitions de l'expression qui précède
- `{n}` : désigne  $n$  occurrences de l'expression qui précède
- `{m,n}` : désigne entre  $m$  et  $n$  répétitions de l'expression qui précède
- `{m,}` : désigne au moins  $m$  répétitions de l'expression qui précède

```
$ egrep 'au?' tg.txt
Il fait beau
il fait chaud
dans cette ville.
$ egrep 'a.*' tg.txt
Il fait beau
il fait chaud
dans cette ville.
```

```
$ egrep 'l{2}' tg.txt
dans cette ville.
$ egrep '[a-Z]{5,}' tg.txt
il fait chaud
dans cette ville.
```

# Expressions régulières : les marqueurs de mots

- Un *mot* est suite de lettres, chiffres ou tirets-bas encadrée par tout autre caractère ou figurant en début ou fin de ligne.
- Le début d'un mot est marqué par la séquence `\<`
- La fin d'un mot est marquée par la séquence `\>`
- `\w` désigne tout caractère autorisé dans un mot, ie. un caractère de la classe `[[:alnum:]]_`

```
$ egrep '\<f tg.txt
Il fait beau
il fait chaud
$ egrep 'it\>' tg.txt
Il fait beau
il fait chaud
$ egrep '\<fait\>' tg.txt
Il fait beau
il fait chaud
```

```
$ egrep '\<\w{4}\>' tg.txt
Il fait beau
il fait chaud
dans cette ville.
$ egrep '\<\w\w\w\>' tg.txt
Il fait beau
il fait chaud
dans cette ville.
$ egrep '\<\w*s\>\s\<\w*\>' tg.txt
dans cette ville.
```

# Expressions régulières : les marqueurs de lignes

- `^` : en début de regex (sinon interprété littéralement), indique que l'expression qui suit débute une ligne
- `$` : en fin de regex (sinon interprété littéralement), indique que l'expression qui précède termine une ligne

```
$ egrep '^I' tg.txt
Il fait beau
$ egrep 'u|\.$' tg.txt
Il fait beau
il fait chaud
dans cette ville.
$ egrep '(u|\.)$' tg.txt
Il fait beau
dans cette ville.
```

# Quelques options de grep

## Quelques options

- `--color` affiche les correspondances en couleur dans chaque ligne
- `-n` indique le numéro de ligne pour chaque ligne trouvée
- `-c` ne retourne que le nombre de lignes trouvées
- `-i` ne différencie pas les majuscules et les minuscules
- `-v` affiche les lignes **sans** correspondance avec la regex

```
$ cat tg.txt
Il fait beau
il fait chaud
dans cette ville.
$ egrep 'cette' tg.txt
dans cette ville.
$ egrep -v 'cette' tg.txt
Il fait beau
il fait chaud
```

```
$ egrep -vc 'cette' tg.txt
2
$ egrep -n 'Il' tg.txt
1:Il fait beau
$ egrep -n -i 'Il' tg.txt
1:Il fait beau
2:il fait chaud
3:dans cette ville.
```

# Recherche avancée de fichiers : la commande `find`

- La commande `find` recherche des fichiers dans l'arborescence du système de fichiers (plus étendue et complexe que `ls`). Par défaut, `find` cherche dans tous les sous-répertoires du répertoire courant de façon récursive.
- `find` utilise des expressions régulières pour établir le modèle des noms de fichier à rechercher.
- La syntaxe générale de la commande `find` est la suivante :

`find repertoires criteres options`

Exemple :

The diagram shows the command `$ find ./Etoiles -name '*.pdf' -exec rm {} \;` with three red ovals highlighting the arguments: `./Etoiles`, `-name '*.pdf'`, and `-exec rm {} \;`. Below each oval is an upward-pointing arrow and a label: `./Etoiles` is labeled "Chemin de répertoire", `-name '*.pdf'` is labeled "Critères de recherche", and `-exec rm {} \;` is labeled "Options".

```
$ find ./Etoiles -name '*.pdf' -exec rm {} \;
```

Chemin de répertoire      Critères de recherche      Options

# La commande `find` : critères de recherche

- `-name` recherche sur le nom de fichier
- `-perm` recherche sur les droits d'accès du fichier
- `-user` recherche sur le propriétaire de fichier
- `-group` recherche sur le groupe auquel appartient le fichier
- `-type` recherche sur le type (`d`=répertoire, `f`= fichier normal)
- `-size` recherche sur la taille du fichier en nombre de blocs (un bloc = 512 octets)
- `-atime` recherche par date de dernier accès au fichier
- `-mtime` recherche par date de dernière modification du fichier
- `-ctime` recherche par date de changement du statut (permissions, nom) du fichier

# La commande `find` : critères de recherche

## Exemples

```
$ find ./Etoiles -name '*.pdf'
```

→ Recherche des fichiers de façon récursive à partir du chemin relatif `./Etoiles` dont le nom se termine par `".pdf"`.

```
$ find /home -perm 741
```

→ Recherche des fichiers de façon récursive à partir du répertoire `/home` (chemin absolu) et dont les permissions sont définies par le code octal `741`.

```
$ find . -mtime 0
```

→ Recherche des fichiers à partir du répertoire courant qui ont été modifiés aujourd'hui.

```
$ find / -user $USER
```

→ Liste tous les fichiers à partir de la racine de l'arborescence du système de fichiers et appartenant à l'utilisateur.

# La commande `find` : combinaison de critères

On peut combiner ces critères avec des opérateurs logiques

- `-a` le "et" logique (AND)
- `-o` le "ou" logique (OR)
- `!` le "non" logique (NOT)
- Le regroupement de la formule logique est encadré par `\( ... \)`

# La commande `find` : combinaison de critères

## Exemples

```
$ find . \(-name "*.txt" -a -mtime 0 \)
```

→ Recherche de façon récursive à partir du répertoire courant des fichiers terminant par l'extension ".txt" (critère `-name "*.txt"`) et modifiés aujourd'hui (critère `-mtime 0`)

→ Attention à bien mettre des espaces comme indiqué ci-dessus sinon la commande ne marchera pas, notamment des espaces après `\(` et avant `\)`.

```
$ find /home \(-name 'fic*' -a -size +1024k \)
```

→ Recherche de fichiers de façon récursive à partir du répertoire `/home` dont le nom commence par les lettres "fic" et dont la taille est supérieure à 1024 kilo octets.

```
$ find / \(-name '*.cpp' -o -size -40c \)
```

→ Recherche à partir de la racine des fichiers qui terminent par l'extension ".cpp" ou qui ont une taille inférieure à 40 octets.

# Les commandes en option de `find`

- Il est possible de limiter la recherche récursive des fichiers dans l'arborescence du système de fichiers avec l'option `-maxdepth`.

Exemple :

```
$ find . -maxdepth 2 -name '*1'
```

→ Affiche la liste des fichiers du répertoire courant dont le nom se termine par 1, en examinant tous les sous-répertoires jusqu'au niveau 2.

- L'option `-ls` permet d'afficher des informations détaillées sur le résultat de la commande `find`.

Exemple :

```
$ find . -name '*.pdf' -ls
```

→ Affiche au format détaillé les fichiers terminant par ".pdf" de façon récursive à partir du répertoire courant.

# Effectuer des actions sur les résultats de la commande `find`

Il est possible d'exécuter des commandes qui vont s'appliquer sur chaque fichier trouvé par `find`.

- `-exec commande` (suivi de `{}` \; ) exécute la commande "commande" sur le résultat de `find`
- `-ok commande` (suivi de `{}` \; ) identique à `-exec commande`, sauf que pour chaque occurrence une confirmation est demandée

# Effectuer des actions sur les résultats de la commande `find`

## Exemples

```
$ find ./Etoiles -name '*.pdf' -exec chmod 777 {} \;
```

→ Passage des droits au code octal 777 des fichiers terminants par ".pdf" de façon récursive dans tous les sous-répertoires à partir du répertoire Etoiles situé dans le répertoire courant.

```
$ find . -name '*.bak' -exec rm {} \;
```

→ Suppression de tous les fichiers terminant par l'extension ".bak" de façon récursive dans tous les sous-répertoires à partir du répertoire courant.

```
$ find $HOME \(-name 'a.out' -o -name '*.o' \) -atime +7 -ok rm {} \;
```

→ Suppression des fichiers a.out et .o situés dans le répertoire personnel de l'utilisateur et accédés il y a plus de 7 jours avec demande de confirmation à l'utilisateur.

# CM Linux : Compléments

# Utiliser efficacement le terminal

## Ouverture/fermeture de terminal

- Lancez un terminal avec la combinaison de touches `<Ctrl> + <Alt> + T`
- Quittez le terminal avec la commande `exit`

## Copier-coller de commandes

- Copiez du texte en le sélectionnant à la souris puis avec la combinaison `<Ctrl> + <Shift> + C`
- Collez le texte dans le terminal avec `<Ctrl> + <Shift> + V`

# Utiliser efficacement le terminal

## Auto-complétion de commandes

- Accédez au répertoire Document en tapant `cd Do` puis la touche `<Tab>`
  - S'il n'existe pas d'autres répertoires dont le nom commence par Do, le terminal complétera
  - L'auto-complétion fonctionne aussi pour les fichiers et noms de commande
  - Elle permet de gagner du temps et d'éviter les fautes de frappe

## Raccourcis clavier

Les raccourcis clavier permettent de gagner du temps et d'éviter de déplacer la main droite vers les flèches. Voir <https://linuxfr.org/users/postroutine/journaux/bash-et-les-raccourcis-clavier> pour une liste de raccourcis assez communs.

# Se connecter à starwars depuis un poste sous Linux/Mac

Voir diapo 113 pour la même manipulation sous Windows 10 (pour les versions antérieures vous devrez utiliser putty ou suivre une autre démarche).

Lancez un terminal puis y exécutez la commande

```
ssh -p 2019 mon_login@leria-etud.univ-angers.fr
```

- Option `-p` en minuscule pour indiquer le numéro de port
- L'identifiant de connexion `mon_login` et le mot de passe demandé correspondent à votre compte LDAP et sont ceux que vous utilisez pour vous connecter à l'ENT et accéder à Zimbra, Moodle, ...

Pour fermer votre session

```
exit
```

# Pour transférer des fichiers entre un poste sous Linux/Mac et le serveur starwars

## Téléversement par SCP

- Copiez le fichier upload dans votre répertoire personnel sur starwars  
`scp -P 2019 upload mon_login@leria-etud.univ-angers.fr:~`

Option `-P` en majuscule pour indiquer le numéro de port

## Téléchargement par SCP

- Copiez le fichier download situé dans votre répertoire personnel sur starwars dans le répertoire courant

```
scp -P 2019 mon_login@leria-etud.univ-angers.fr:~/download .
```

## Montage par SSHFS

- Montez votre répertoire personnel sur starwars dans un dossier local (ici monDossier)

```
sshfs -p 2019 mon_login@leria-etud.univ-angers.fr: ./monDossier
```

# Utiliser bash depuis Windows 10

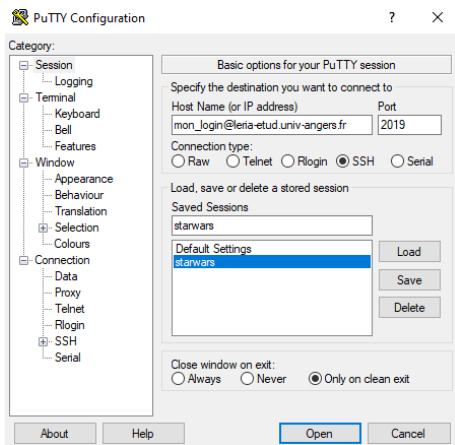
Sous Windows 10, vous avez accès à un shell Linux en suivant les explications des liens ci-dessous

- <https://www.youtube.com/watch?v=CyG16N3GJWo>
- <https://www.youtube.com/watch?v=Xnt6XhWw0SE>
- <https://blog.ineat-group.com/2020/02/utiliser-le-terminal-bash-natif-dans-windows-10/>

Si vous souhaitez lancer des applications graphiques (Firefox, Gedit, Kate, ...) depuis le shell Linux, vous devrez installer un serveur X sur votre machine et ajouter la ligne `export DISPLAY=:0` à votre `.bashrc`.

# Se connecter à starwars depuis un poste sous Windows

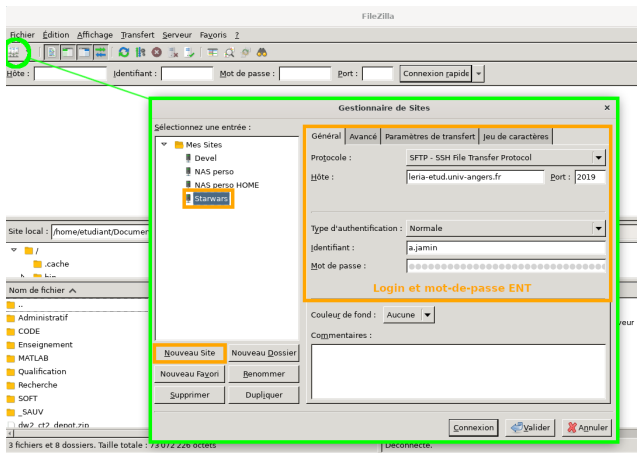
Avec le logiciel Putty (<https://www.putty.org/>)



Bien préciser l'host name et le port puis cliquer sur open.

# Pour transférer des fichiers entre un poste sous Windows et starwars

Avec le logiciel Filezilla



# Se connecter à starwars sous Windows 10

## Avec le client SSH de Windows 10

### ❶ Vérifiez que OpenSSH est installé :

- ❶ Cherchez powershell dans vos applications (touche Windows puis "powershell")
- ❷ Clic droit sur l'icône de powershell : "Exécuter en tant qu'administrateur"
- ❸ Lancez la commande :  
`Get-WindowsCapability -Online | Where{ $_.Name -like 'OpenSSH.Client*' }`
- ❹ Si le "State" d'OpenSSH n'est pas "Installed" alors installez le avec la commande :

`Add-WindowsCapability -Online -Name OpenSSH.Client`

### ❷ Ouvrez le powershell (plus besoin d'être en administrateur) et exécutez la commande :

`ssh -p 2019 mon_login@leria-etud.univ-angers.fr`

# Pour tester les exemples du cours

- 1 A chaque section du cours correspond une archive des fichiers tests utilisés, p. ex. `cm-linux-1.tgz` contient les fichiers de la première section du CM. Téléchargez l'archive de votre choix sur votre ordinateur, transférez-la dans votre répertoire personnel sur le serveur `starwars` puis connectez-vous à `starwars`. Par exemple, sous Linux :  

```
scp -P 2019 cm-linux-1.tgz mon_login@leria-etud.univ-angers.fr:~
ssh -p 2019 mon_login@leria-etud.univ-angers.fr
```
- 2 Dans votre répertoire personnel sur `starwars`, décompressez l'archive  

```
tar xvf cm-linux-1.tgz
```
- 3 Déplacez-vous dans le sous-répertoire créé  

```
cd cm-linux-1
```

Vous pouvez dorénavant reproduire les exemples de commandes donnés dans cette section du cours

# Commandes générales sur les fichiers

## Nom, chemin et type d'un fichier

- `basename` donne le nom du fichier (sans son chemin)
- `dirname` en donne le chemin (sans le nom du fichier)
- `file` donne des informations sur le type du fichier

## Exemple

```
$ basename SOLEIL.jpg
```

```
SOLEIL.jpg
```

```
$ dirname SOLEIL.jpg
```

```
.
```

```
$ file SOLEIL.jpg
```

```
SOLEIL.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density
1x1, segment length 16, baseline, precision 8, 2400x2400, frames 3
```

## Liens vers les fichiers – liens physiques

On crée un lien physique vers un fichier avec la commande `ln`

- Un lien physique n'est pas un « raccourci » vers un fichier, car la destruction de l'original **n'affecte pas le lien physique**.
- Les données du fichier ne sont pas détruites tant qu'il existe un lien physique vers le fichier.
- Nombre de liens physiques visibles avec `stat file1`

```
$ more file1
abcdef
$ ln file1 file2
$ more file2
abcdef
$ rm file1
$ more file2
abcdef
```

## Liens vers les fichiers – liens symboliques

On crée un lien symbolique vers un fichier avec la commande `ln -s`

- Un lien symbolique n'est qu'un « raccourci » vers un fichier, la destruction du fichier original rend le lien symbolique inopérant.

```
$ more file1
abcdef
$ ln -s file1 file2
$ more file2
abcdef
$ rm file1
$ more file2
file2 : No such file or directory
```

# Taille de fichiers et de répertoires

L'espace disque utilisé par un fichier ou un répertoire est donné par la commande `du`

`du [-ahks] [-B] [FILE]`

```
$ ls
```

```
Etoiles fichier1 Système_Solaire
```

```
$ du -B 1
```

```
176128 ./Etoiles
```

```
28672 ./Système_Solaire
```

```
208896 .
```

→ le répertoire `Etoiles` occupe 176128 octets (bytes = 8 bits)

```
$ du -B 1KB Etoiles
```

```
176 ./Etoiles
```

→ le répertoire `Etoiles` occupe 176 kilo-octets (1KB = 1000 bytes)

```
$ du -B 1K Etoiles
```

```
172 ./Etoiles
```

→ le répertoire `Etoiles` occupe 172 kébi-octets (1K = 1024 bytes)

# Les variables en bash

## Les variables

- Comme tout programme, `bash` peut manipuler des variables
- Les variables facilitent l'interaction avec l'OS et l'écriture de commandes avancées
- Elles peuvent être déclarées, définies, exportées, utilisées et supprimées en ligne de commande ou dans des scripts `bash`
- Leurs types de valeur (entiers, chaînes de caractères, tableaux ...) dépend des valeurs/expressions qui leurs sont affectées
- Certaines variables pré-existent : les variables dites d'environnement

# Manipulation de variables

## Opérations élémentaires sur les variables en bash

- `var=val` : affecte la valeur `val` à la variable `var`
- `echo $var` : affiche de la variable `var` (préfixez par `$`)
- `unset var` : supprime la variable `var`

## Exemple

```
$ myx=12; echo $myx
```

```
12
```

```
$ unset myx; echo $myx
```

```
$ TABLE=("A" "B" "C")
```

```
$ echo ${TABLE[0]}
```

```
A
```

```
$ TABLE[0]="Z"
```

```
$ echo ${TABLE[0]}
```

```
Z
```

# Calcul sur les variables

```
$ x=12
```

```
$ echo $((x+8)); echo $((2*x)); echo $((x/2))
```

20

24

6

→ addition, multiplication et division entière avec une constante

```
$ x=AB; y=CD; r=xy; echo $r
```

ABCD

→ concaténation

```
$ z=12; t=13; r=zt; echo $r
```

1213

→ concaténation

```
$ r=$((z+t)); echo $r
```

25

→ l'addition, merci

```
$ r=$((x+t)); echo $r
```

13

→ addition “mixte” : chaîne transtypée à l'entier 0

# Les variables d'environnement

Elles existent dans le shell et sont accessibles à tous les processus

- **USER** : le login (identifiant) de l'utilisateur
- **HOME** : le chemin absolu du répertoire personnel
- **PWD** : le chemin du répertoire courant
- **LANG** : la langue utilisée par bash
- **PATH** : la liste de répertoires où le shell recherche les commandes
- **SECONDS** : le nombre de secondes depuis le lancement du shell courant
- **COLUMNS** et **LINES** : largeur et hauteur du terminal en caractères

## Affichage et export

- On affiche une variable en la préfixant par **\$** avec **echo \$HOME**
- La commande **env** les affiche toutes
- On peut exporter ses propres variables dans l'environnement avec **export \$myX**

# Exemple d'utilisation des variables d'environnement

```
$ echo $USER
```

```
my.login
```

```
$ echo $HOME
```

```
/home/m/y/my.login
```

```
$ echo $PWD
```

```
/home/m/y/my.login/Demolinux
```

```
$ echo $LANG
```

```
en_US.UTF-8
```

```
$ echo $PATH
```

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr
```

→ bash cherchera d'abord la commande dans /usr/local/sbin et, s'il ne la trouve pas, dans /usr/local/bin, et ainsi de suite

```
$ export PATH=$PATH:~/prog
```

→ ajout du répertoire ~/prog en queue de PATH

→ les commandes ne seront recherchées dans ~/prog que si elles n'ont pas été trouvées dans les répertoires le précédant dans \$PATH

```
$ export PATH=~/prog:$PATH
```

→ ajout du répertoire ~/prog en tête de PATH (non recommandé)

# Traitement des chaînes de caractères

Une chaîne de caractères s'encadre de 3 façons différentes

- Avec apostrophes `'` (alias guillemets simples)
- Avec guillemets `"` (alias guillemets doubles)
- Avec accents graves ``` (alias backtick, backquote)

Une commande interprète une chaîne différemment selon son encadrement

- Les apostrophes inhibent tous les caractères spéciaux  
`echo '$USER'` renvoie `$USER`
- Les guillemets inhibent tous les caractères spéciaux sauf `$`, `\` et ```  
`echo "$USER"` renvoie `my.login`
- Les accents graves exécutent la chaîne comme s'il s'agissait d'une commande : le résultat sert d'argument à la commande englobante  
`echo ` $USER `` renvoie `my.login: command not found`  
`echo `whoami`` renvoie `my.login`

# Les alias

Un **alias** est un nom court et facilement mémorisable pour désigner une commande complexe et fréquemment utilisée

- Un alias s'invoque dans un shell comme n'importe quelle commande et exécute la commande qu'il désigne

Les commandes **alias** et **unalias** créent et suppriment des alias

```
$ alias
```

→ listing des alias existants

```
$ alias menage="rm *.bak *.old; clear"
```

```
$ alias sshsw='ssh -p 2019 my.login@leria-etud.univ-angers.fr'
```

→ création des alias menage et sshd

```
$ menage
```

...

→ utilisation de l'alias menage

```
$ unalias menage
```

→ suppression de l'alias menage

# Personnalisation de l'environnement bash

Les shells sont personnalisables via le fichier de configuration

## `.bash_profile`

- Il se situe dans le dossier `$HOME` de l'utilisateur ayant lancé la session
- Il est lu et exécuté à chaque création de session

Il existe d'autres fichiers de "configuration" comme `.bash_aliases`

- Si on exécute `alias la='ls -a'`, l'alias `la` ne sera plus valide une fois le terminal fermé au lancement d'un nouveau terminal
- Si on souhaite le conserver, il suffit de placer la commande dans le fichier `.bash_aliases` (ou `.bash_profile`) avec la commande  

```
echo "alias la='ls -a'" >> .bash_aliases
```
- L'alias `la` n'est pas immédiatement utilisable dans le terminal : pour ce faire, il faut réexécuter le fichier `.bash_aliases` avec la commande `source .bash_aliases`

# Concaténation avec paste

Le filtre `paste` concatène les lignes de même numéro des fichiers indiqués

- L'appel se fait simplement comme suit, même si les fichiers n'ont pas le même nombre de lignes  
\$ `paste fichier1 fichier2`
- Cette fonction est utile en particulier car la commande `cut` ne permet pas d'inverser l'ordre des champs d'un fichier.

## Exemple d'utilisation de paste

```
$ cat city
```

```
Angers 49000 FRANCE
```

```
Nantes 44000 FRANCE
```

```
$ cut -d ' ' -f 2,1,3 city
```

```
Angers 49000 FRANCE
```

```
Nantes 44000 FRANCE
```

→ Aucun changement !

```
$ paste <(cut -d ' ' -f2 city) <(cut -d ' ' -f1,3 city)
```

```
49000 Angers FRANCE
```

```
44000 Nantes FRANCE
```

→ On fait à part l'extraction du deuxième champ, ainsi que des premier et troisième champs, et on colle le tout ligne par ligne avec la commande `paste`.

# Décomptes avec `wc`

La commande `wc` (word count) permet de compter le nombre de lignes, de mots et de caractères dans un ou plusieurs fichiers

- `-l` compte le nombre de lignes
- `-c` compte le nombre de caractères
- `-w` compte le nombre de mots

# Décomptes avec wc

## Exemples

```
$ wc file1.txt file2.txt
```

```
2 5 22 file1.txt
```

```
5 5 114 file2.txt
```

```
7 10 136 total
```

→ Ceci signifie qu'il y a 2 lignes, 5 mots et 22 caractères dans file1.txt et qu'il y a 5 lignes, 5 mots et 114 caractères dans file2.txt. Le total sur les deux fichiers est ensuite affiché.

```
$ wc -lc file1.txt
```

```
2 22 file1.txt
```

→ La commande renvoie dans ce cas seulement le nombre de lignes et de caractères contenus dans file1.txt.

# Comparaison de fichiers avec comm

La commande `comm` compare deux fichiers dont les lignes sont supposées triées

- `comm fic1 fic2` affiche trois sorties :
  - les lignes propres à `fic1`, colonne de gauche
  - les lignes propres à `fic2`, colonne du centre
  - les lignes communes à `fic1` et `fic2`, colonne de droite
- L'option `-1` supprime la première sortie
- L'option `-2` supprime la seconde sortie
- L'option `-3` supprime la troisième sortie
- Les lignes affichées sont triées.

## Exemple d'utilisation de comm

```
$ cat fichier1
```

```
Adeline :0644234452 :Paris
```

```
Catherine :0613344433 :Nantes
```

```
Daniel :0698334432 :Angers
```

```
Mounir :0666342233 :Cholet
```

```
$ cat fichier2
```

```
Catherine :0613344433 :Nantes
```

```
Céline :0466342233 :Marseille
```

```
Daniel :0698334432 :Angers
```

```
Mounir :0666342233 :Cholet
```

```
$ comm -12 fichier1 fichier2
```

```
Catherine :0613344433 :Nantes
```

```
Daniel :0698334432 :Angers
```

```
Mounir :0666342233 :Cholet
```

# Comparaison de fichiers avec `cmp`

La commande `cmp` indique la première différence entre deux fichiers

- `cmp fic1 fic2` ne renvoie rien si `fic1` et `fic2` sont identiques
- S'ils sont différents, `cmp` indique le numéro du premier octet qui diffère, ainsi que le numéro de la ligne où il se trouve

## Exemple d'utilisation de cmp

```
$ cat fichier1
```

```
Daniel:0698334432:Angers
```

```
Adeline:0644234452:Paris
```

```
Mounir:0666342233:Cholet
```

```
Catherine:0613344433:Nantes
```

```
$ cat fichier2
```

```
Daniel:0698334432:Angers
```

```
Céline:0466342233:Marseille
```

```
Mounir:0666342233:Cholet
```

```
Catherine:0613344433:Nantes
```

```
$ cmp fichier1 fichier2
```

```
fichier1 fichier2 sont différents: octet 26, ligne 2
```

# Comparaison de fichiers avec diff

Le filtre diff indique **toutes** les différences entre deux fichiers

- La commande `diff fic1 fic2` indique toutes les modifications à apporter à `fic1` pour le rendre identique à `fic2` (rien n'est retourné si ces fichiers sont identiques)
- Le résultat utilise les symboles avec la signification suivante :
  - `<` désigne une ligne de `fic1`
  - `>` désigne une ligne de `fic2`
  - `a` ajouter, `c` changer, `d` supprimer

Ainsi,

- `5a1` signifie « **ajouter** en **5ème** position dans `fic1` la ligne **1** de `fic2` »
- `3c2` signifie « **changer** la ligne **3** de `fic1` et la remplacer par la ligne **2** de `fic2` »
- `2d1` signifie « **supprimer** la ligne **2** de `fic1`, le compteur de ligne redescendra à **1** »

## Exemple d'utilisation de diff

```
$ cat fichier1
Daniel:0698334432:Angers
Adeline:0644234452:Paris
Mounir:0666342233:Cholet
Catherine
:0613344433:Nantes
```

```
$ cat fichier2
Daniel:0698334432:Angers
Céline
:0466342233:Marseille
Mounir:0666342233:Cholet
```

```
$ diff fichier1 fichier2
2c2
< Adeline:0644234452:Paris

> Céline :0466342233:Marseille
4d3
< Catherine :0613344433:Nantes
```

Pour que fichier1 soit identique à fichier2, il faut que :

- la ligne 2 de fichier1 soit **changée** en la ligne 2 de fichier2
- la ligne 4 de fichier1 soit **supprimée**

# Filtre split

Le filtre `split` découpe un fichier en plusieurs fichiers plus petits

C'est utile par exemple pour envoyer un fichier volumineux en plusieurs fois par courrier électronique.

- `-l` indique le nombre de lignes à copier dans chaque fichier créé
- `-b` indique la taille maximale de chaque fichier créé

La commande suivante

```
$ split -l 10 monfichier prefixe
```

crée les fichiers `prefixeaa`, `prefixeab`, `prefixeac`, ... qui contiendront tous 10 lignes. `prefixeaa` contient les 10 premières lignes de `fichier`, `prefixeab` les 10 lignes suivantes et ainsi de suite.

# Filtre split – le retour de cat

- Quand on a réceptionné les fichiers `prefixeaa`, ..., `prefixeac` créés par `split`, comment reconstituer le fichier d'origine ?
- On utilise la commande `cat` dont la fonction première est de `concaténer` des fichiers pour en former un seul.
- Reconstitution du fichier sous le nom `file` :

```
cat prefixeaa prefixeab prefixeac > file
```

Ou mieux :

```
cat prefixe* > file
```

- Ensuite, on peut supprimer les fichiers `prefixeaa`, `prefixeab` etc, dont on n'a plus besoin

```
rm prefixe*
```

# Différences entre grep et egrep

En syntaxe basique BRE - avec la commande `grep` sans l'option `E` -, il faut échapper les caractères suivants pour qu'ils ne soient pas interprétés littéralement

? + { } | ( )

Par exemple, les 2 commandes suivantes sont équivalentes

```
$ egrep '(ai)|(et)' tg.txt
```

```
Il fait beau
```

```
il fait chaud
```

```
dans cette ville.
```

```
$ grep '\(ai\)\\|\(et\)\' tg.txt
```

```
Il fait beau
```

```
il fait chaud
```

```
dans cette ville.
```